CONSTRAINT-BASED PROBLEM SOLVING

Scenario: A university must schedule five courses—Artificial Intelligence (A), Database Systems (B), Computer Networks (C), Operating Systems (D), and Software Engineering (E)—into three exam slots: Slot1, Slot2, and Slot3. Conflicting courses must be assigned different slots.

a) PEAS Description

PEAS Element	Description
Performance Measure	Conflict-free timetable, all courses scheduled, all constraints satisfied, and efficient use of the three slots.
Environment	University examination scheduling environment containing 5 courses, 3 available exam slots, and student-enrollment conflicts.
Actuators	The scheduling agent assigns or changes a course's exam slot and produces the final timetable.
Sensors	Course list, available slots, conflict pairs, current assignments, and remaining domains of unassigned courses.

b) Suitable Intelligent Agent Type

A goal-based agent is best suited for this problem. The agent has a clear goal: produce a conflict-free exam timetable satisfying all constraints. It evaluates possible assignments and uses search/backtracking to reach a valid goal state. A simple reflex agent is not suitable because it cannot effectively reason about future conflicts. A model-based agent could represent the problem, but the main requirement here is goal-directed search. A utility-based agent is unnecessary because no preference or cost function is specified.

c) CSP Formulation and Constraint Graph

Variables: A, B, C, D, E.

Domains: D(A) = D(B) = D(C) = D(D) = D(E) = {Slot1, Slot2, Slot3}.

Constraints: Each listed conflicting pair must receive different slots (≠): A≠B, A≠C, B≠C, B≠D, C≠D, C≠E, D≠E.

Constraint graph: Vertices = {A, B, C, D, E}; Edges = {AB, AC, BC, BD, CD, CE, DE}.

Adjacency list:

A → B, C

B → A, C, D

C → A, B, D, E

D → B, C, E

E → C, D

d) Plain Backtracking Search

Python Program

File Edit Format Run Options Window Help

```
courses = ['A', 'B', 'C', 'D', 'E']
slots = ['Slot1', 'Slot2', 'Slot3']
conflicts = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D', 'E'],
    'D': ['B', 'C', 'E'],
    'E': ['C', 'D']
}

assignment = {}
def is_safe(course, slot):
    for neighbor in conflicts[course]:
        if neighbor in assignment and assignment[neighbor] == slot:
            return False
    return True
def backtracking():
    if len(assignment) == len(courses):
        return True
    course = courses[len(assignment)]
    for slot in slots:
        print("Trying", course, "=", slot)
        if is_safe(course, slot):
            assignment[course] = slot
            if backtracking():
                return True
            print("Backtracking from", course, "=", slot)
            del assignment[course]
        else:
            print("Conflict! Cannot assign", course, "=", slot)
    return False
if backtracking():
    print("\nFinal Timetable:")
    for course in courses:
        print(course, "->", assignment[course])
else:
    print("No solution exists.")
```

Output

```
Trying A = Slot1
Trying B = Slot1
Conflict! Cannot assign B = Slot1
Trying B = Slot2
Trying C = Slot1
Conflict! Cannot assign C = Slot1
Trying C = Slot2
Conflict! Cannot assign C = Slot2
Trying C = Slot3
Trying D = Slot1
Trying E = Slot1
Conflict! Cannot assign E = Slot1
Trying E = Slot2

Final Timetable:
A -> Slot1
B -> Slot2
C -> Slot3
D -> Slot1
E -> Slot2
>>>
```

e) Backtracking with Forward Checking

Python Program

```
courses = ['A', 'B', 'C', 'D', 'E']
slots = ['Slot1', 'Slot2', 'Slot3']
conflicts = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D', 'E'],
    'D': ['B', 'C', 'E'],
    'E': ['C', 'D']
}
domains = {
    'A': ['Slot1', 'Slot2', 'Slot3'],
    'B': ['Slot1', 'Slot2', 'Slot3'],
    'C': ['Slot1', 'Slot2', 'Slot3'],
    'D': ['Slot1', 'Slot2', 'Slot3'],
    'E': ['Slot1', 'Slot2', 'Slot3']
}
assignment = {}
def forward_check(course, slot, current_domains):
    new_domains = {
        c: current_domains[c].copy()
        for c in courses
    }
    new_domains[course] = [slot]
    for neighbor in conflicts[course]:
        if neighbor not in assignment:
            if slot in new_domains[neighbor]:
                new_domains[neighbor].remove(slot)
            if len(new_domains[neighbor]) == 0:
                return None
    return new_domains
def backtracking_fc(current_domains):
    if len(assignment) == len(courses):
        return True
    course = courses[len(assignment)]
    for slot in current_domains[course]:

        print("\nTrying", course, "=", slot)
        conflict = False
        for neighbor in conflicts[course]:
            if neighbor in assignment:
                if assignment[neighbor] == slot:
                    conflict = True
                    break
        if conflict:
            print("Conflict! Cannot assign", course, "=", slot)
            continue
        new_domains = forward_check(
            course, slot, current_domains
        )
        if new_domains is None:
            print("Forward checking failed for", course, "=", slot)
            continue
        print("Domains after pruning:")
        for c in courses:
            if c not in assignment and c != course:
                print(c, ":", new_domains[c])
        assignment[course] = slot
        if backtracking_fc(new_domains):
            return True
        print("Backtracking from", course, "=", slot)
        del assignment[course]
    return False
print("BACKTRACKING WITH FORWARD CHECKING")
print("----------------------------------")
if backtracking_fc(domains):
    print("\nFinal Timetable:")
    for course in courses:
        print(course, "->", assignment[course])
else:
    print("No solution exists.")
```

Output

```
BACKTRACKING WITH FORWARD CHECKING
----------------------------------

Trying A = Slot1
Domains after pruning:
B : ['Slot2', 'Slot3']
C : ['Slot2', 'Slot3']
D : ['Slot1', 'Slot2', 'Slot3']
E : ['Slot1', 'Slot2', 'Slot3']

Trying B = Slot2
Domains after pruning:
C : ['Slot3']
D : ['Slot1', 'Slot3']
E : ['Slot1', 'Slot2', 'Slot3']

Trying C = Slot3
Domains after pruning:
D : ['Slot1']
E : ['Slot1', 'Slot2']

Trying D = Slot1
Domains after pruning:
E : ['Slot2']

Trying E = Slot2
Domains after pruning:

Final Timetable:
A -> Slot1
B -> Slot2
C -> Slot3
D -> Slot1
E -> Slot2
>>>
```

f) Comparison of Plain Backtracking and Forward Checking

Aspect	Plain Backtracking	Forward Checking	Advantage
Domain handling	Checks conflicts when assigning the next variable.	Also removes assigned values from neighboring unassigned domains.	Forward Checking
Detection of failure	Failure may be detected later.	Detects an empty domain immediately.	Forward Checking
Search effort	May explore more assignments.	Prunes inconsistent future values.	Forward Checking
Backtracking	Can require more backtracks.	Usually requires fewer backtracks.	Forward Checking
Memory	Lower.	Slightly higher due to maintained domains.	Plain Backtracking
Efficiency	Good for small CSPs.	Generally better for constrained CSPs.	Forward Checking

Conclusion

The university exam scheduling problem is successfully modeled as a Constraint Satisfaction Problem. Plain backtracking finds a valid timetable by trying assignments and backtracking after conflicts. Forward checking improves the search by pruning unavailable slot values immediately after each assignment. Therefore, forward checking generally performs fewer unnecessary searches and is more efficient for larger or more constrained scheduling problems.